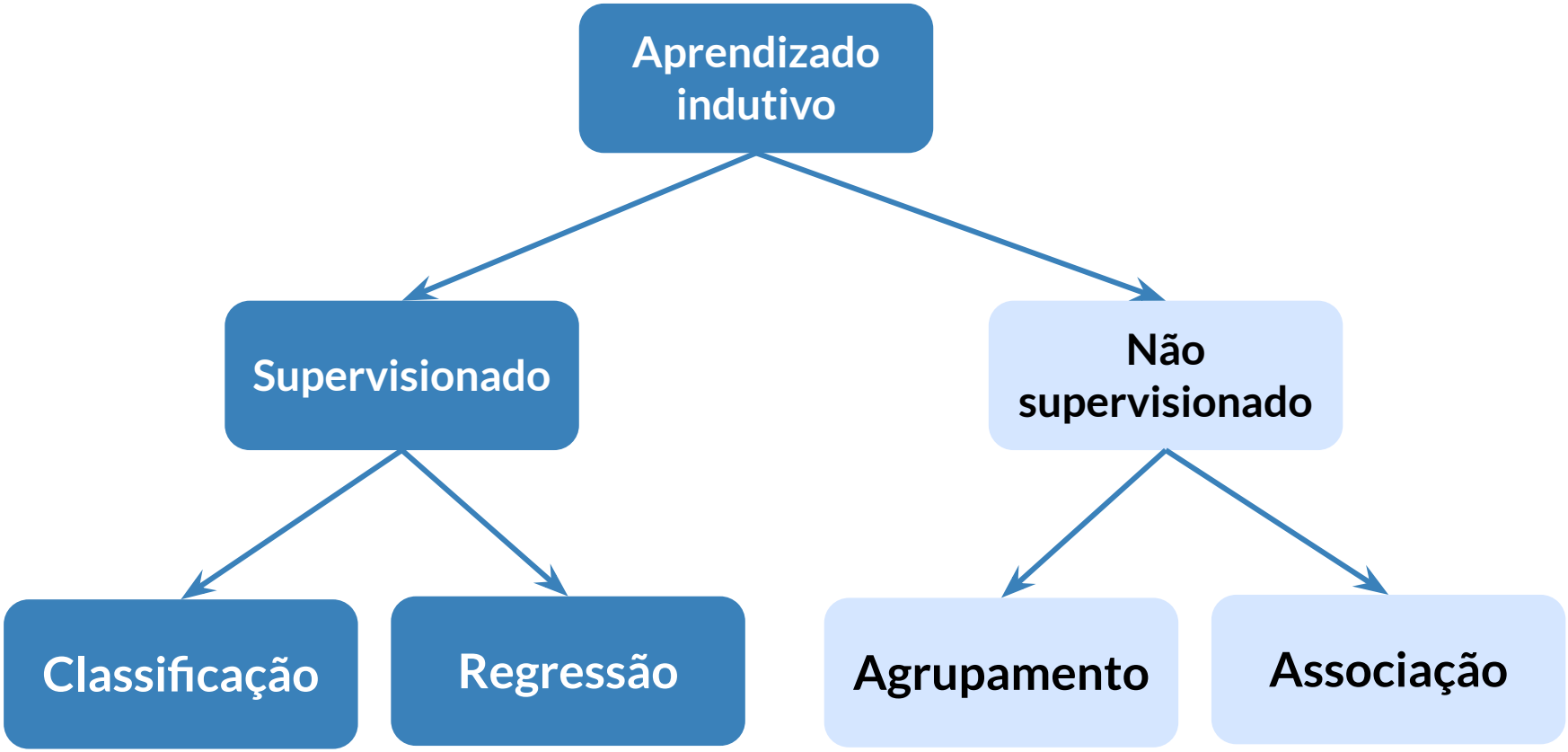
Árvores de Decisão
Cálculo do Ganho

Gabriel P. Oliveira

Inteligência Artificial
2026/1

Árvores de Decisão



Árvores de Decisão/Regressão são algoritmos de aprendizado preditivo

ADs – Como gerar?

Na maioria dos algoritmos indutores de árvores de decisão, as funções de divisão discreta são **univariadas**, isto é, um nó interno é dividido **de acordo com o valor de um único atributo**.

Consequentemente, o indutor procura **o melhor atributo sobre o qual realizar a divisão**.

ADs – Como gerar?

Lembrando:

- Busca gulosa foi feita para **minimizar a profundidade** da árvore final
- A ideia é escolher **o atributo que vá o mais longe possível** na tentativa de fornecer uma classificação exata dos exemplos

ADs – Como gerar?

Um atributo **perfeito** divide os exemplos em conjuntos, cada um dos quais será todo positivo ou negativo (folhas da árvore)

Exemplo – Esperar uma mesa?

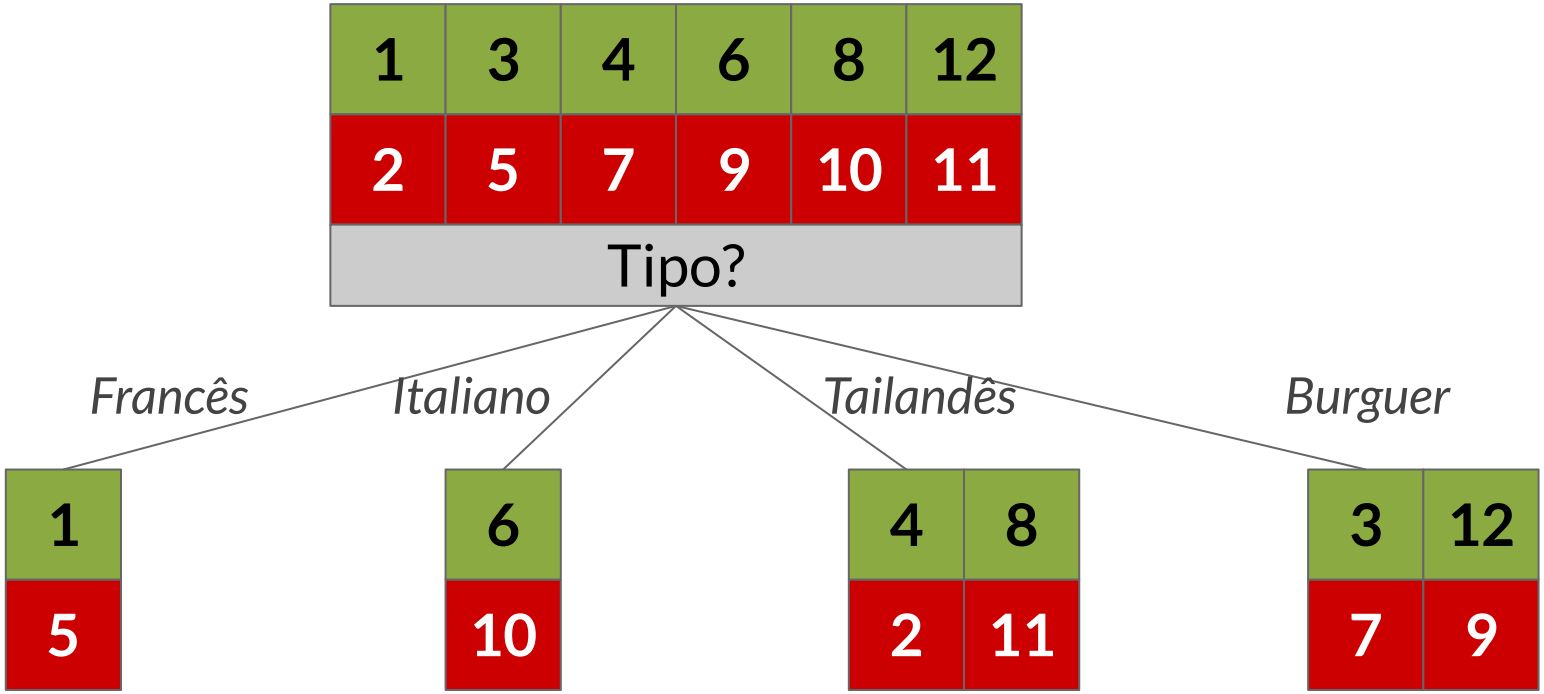
Exemplo	Alt.	Bar	Sex/Sab	Fome	Clientes	Pr.	Chuva	Res.	Tipo	Estim.	VaiEsperar
x1	Sim	Não	Não	Sim	Alguns	$$$	Não	Sim	Francês	0-10	Sim
x2	Sim	Não	Não	Sim	Cheio	$	Não	Não	Tailandês	30-60	Não
x3	Não	Sim	Não	Não	Alguns	$	Não	Não	Burguer	0-10	Sim
x4	Sim	Não	Sim	Sim	Cheio	$	Sim	Não	Tailandês	10-30	Sim
x5	Sim	Não	Sim	Não	Cheio	$$$	Não	Sim	Francês	>60	Não
x6	Não	Sim	Não	Sim	Alguns	$$	Sim	Sim	Italiano	0-10	Sim
x7	Não	Sim	Não	Não	Nenhum	$	Sim	Não	Burguer	0-10	Não
x8	Não	Não	Não	Sim	Alguns	$$	Sim	Sim	Tailandês	0-10	Sim
x9	Não	Sim	Sim	Não	Cheio	$	Sim	Não	Burguer	>60	Não
x10	Sim	Sim	Sim	Sim	Cheio	$$$	Não	Sim	Italiano	10-30	Não
x11	Não	Não	Não	Não	Nenhum	$	Não	Não	Tailandês	0-10	Não
x12	Sim	Sim	Sim	Sim	Cheio	$	Não	Não	Burguer	30-60	Sim

ADs – Como gerar?

Um atributo **perfeito** divide os exemplos em conjuntos, cada um dos quais será todo positivo ou negativo (folhas da árvore)

Exemplo – Esperar uma mesa?

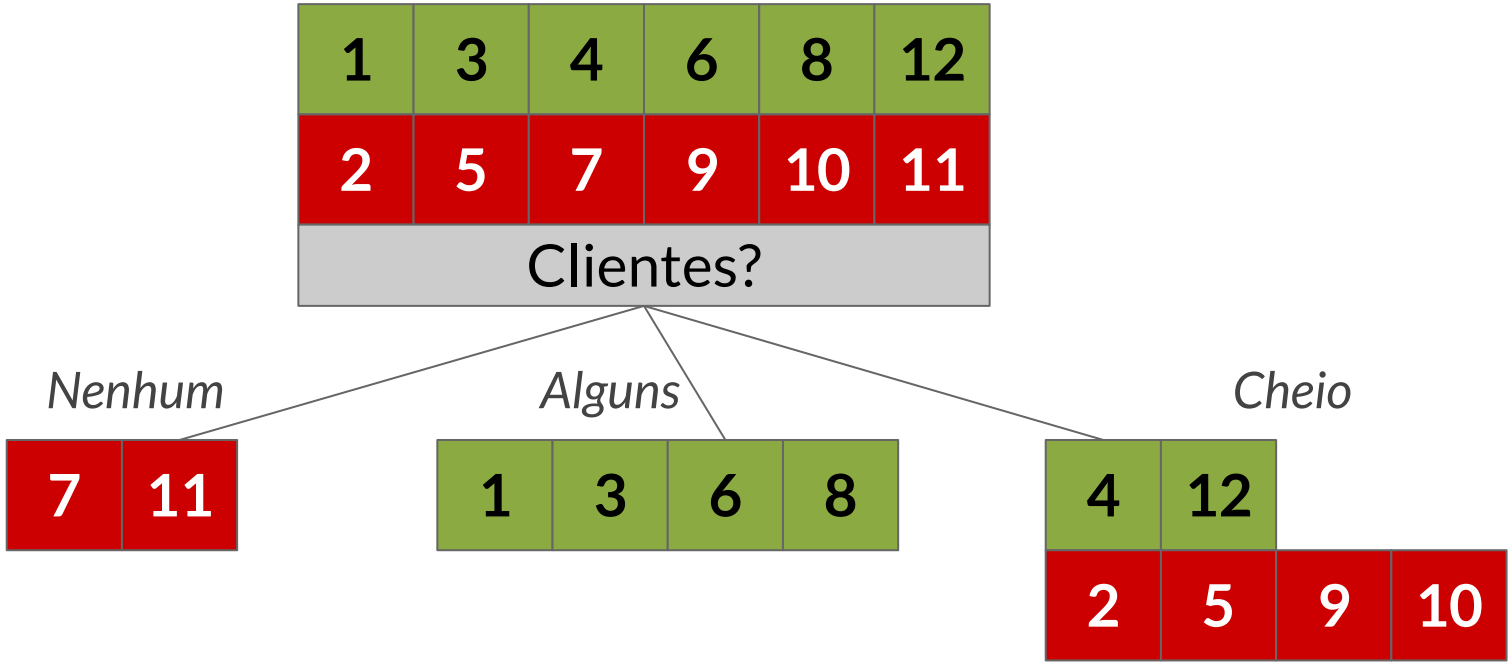
Vamos testar o atributo "**Tipo**"?



"**Tipo**" é um **atributo fraco** porque cada resultado tem o mesmo número de exemplos positivos e negativos.

Exemplo – Esperar uma mesa?

Vamos testar o atributo "**Clientes**"?



"**Cliente**" é um **atributo bastante importante**.

Árvores de Decisão

função APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO(*exemplos, atributos, exemplos-pais*)
retorna uma árvore de decisão
  se exemplos é vazio **então retornar** VALOR-DA-MAIORIA (*exemplos_pais*)
  senão se todos os *exemplos* têm a mesma classificação **então retornar** a classificação
  senão se *atributos* é vazio **então retornar** VALOR-DA-MAIORIA(*exemplos*)
  senão
    $A \leftarrow \text{argmax}_{a \in \text{atributes}}$ IMPORTÂNCIA (a, *exemplos*)
    árvore ← uma nova árvore de decisão com teste de raiz A
    para cada valor v_k de A **faça**
      exs ← $\{e : e \in \textit{exemplos}$ **e** $e.A = v_k\}$
      subárvore ← APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO (*exs, atributos* — A, *exemplos*)
      adicionar uma ramificação à árvore com rótulo ($A = v_k$) e subárvore *subárvore*
    retornar *árvore*

Precisamos de uma **medida formal** para "muito importante" e "inútil"

Árvores de Decisão

Como avaliar **matematicamente** o ganho de cada atributo?

- Ou seja, como decidir o atributo mais importante?

1.

Entropia e Ganho de Informação

O que são e como calcular?

Entropia

A entropia é uma medida da **incerteza** de uma variável aleatória e vem da **teoria da informação.**

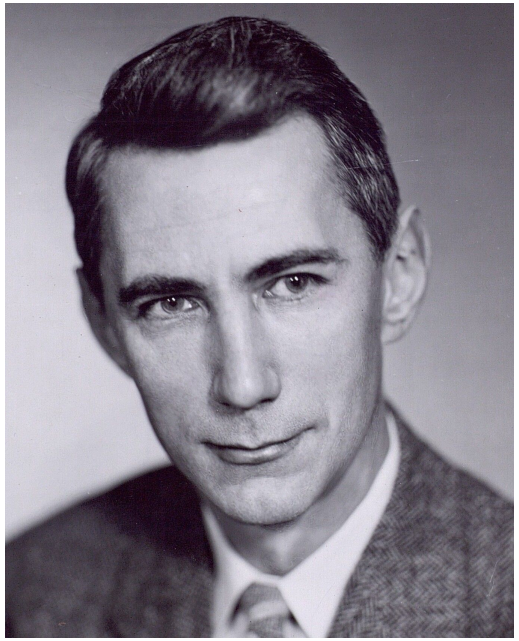
- **Física/Química:** quantidade de desordem associada a um sistema
- **Teoria da informação:** grau de desordem de um conjunto de dados

Entropia

A entropia é uma medida da **incerteza** de uma variável aleatória e vem da **teoria da informação**

Proposta por **Claude Shannon** no artigo *"A Mathematical Theory of Communication"* (1948)

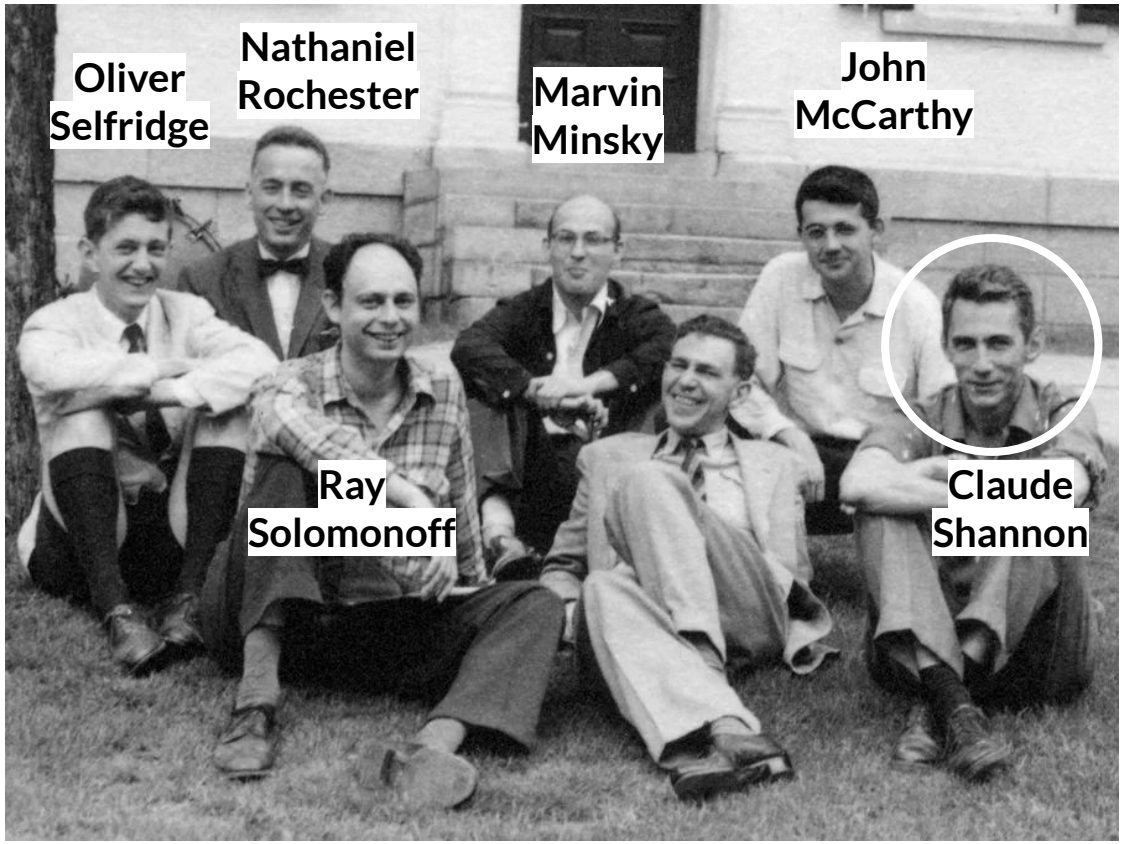
Veja o artigo:
https://ieeexplore.ieee.org/document/6773024



Claude E. Shannon
(1916-2001)

Entropia

1956 – **1º Workshop de IA** – Dartmouth College (EUA)



Organizadores e alguns dos participantes do workshop em 1956 (IEEE Spectrum)

Entropia

Shannon queria medir a **quantidade média de informação** (ou surpresa) ao de um evento aleatório X

- A noção de **informação** pode ser entendida como a resposta a uma pergunta
- A quantidade de informações contidas na resposta depende do **conhecimento prévio** do indivíduo
- Quanto menos se sabe anteriormente, mais informações são fornecidas

Entropia

Shannon queria medir a **quantidade média de informação** (ou surpresa) ao de um evento aleatório X

- Um evento **certo** (p(x) = 1) não traz informação nova, portanto a entropia é **zero**
- Um evento com **probabilidade baixa** traz **mais informação** quando ocorre (entropia maior)

Exemplos?

Entropia

Shannon queria medir a **quantidade média de informação** (ou surpresa) ao de um evento aleatório X

- Um evento **certo** (p(x) = 1) não traz informação nova, portanto a entropia é **zero**
- Um evento com **probabilidade baixa** traz **mais informação** quando ocorre (entropia maior)

Exemplo: Saber que "vai chover no deserto do Saara" (raro) traz mais informação do que "vai chover na Amazônia"(evento esperado)

Entropia

Shannon queria medir a **quantidade média de informação** (ou surpresa) ao de um evento aleatório X

$$H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)}$$

Entropia

Shannon queria medir a **quantidade média de informação** (ou <u>surpresa</u>) ao de um evento aleatório X

Entropia normalmente representada pela letra *H*

$$\boxed{H}(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)}$$

Entropia

Shannon queria medir a **quantidade média de informação** (ou <u>surpresa</u>) ao de um evento aleatório X

Variável aleatória com valores v_k
Conjunto de classes

$$H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)}$$

Entropia

Shannon queria medir a **quantidade média de informação** (ou <u>surpresa</u>) ao de um evento aleatório X

$$H(V) = \sum_k \boxed{P(v_k)} \log_2 \frac{1}{P(v_k)}$$

Probabilidade do valor v_k
Cada classe (e.g., Sim, Não)

Entropia

Shannon queria medir a **quantidade média de informação** (ou <u>surpresa</u>) ao de um evento aleatório X

$$H(V) = \sum_k P(v_k) \boxed{\log_2 \frac{1}{P(v_k)}}$$

Quantidade de informação
Cada classe (e.g., Sim, Não)

Entropia

Shannon queria medir a **quantidade média de informação** (ou <u>surpresa</u>) ao de um evento aleatório X

$$H(V) = \boxed{\sum_k} P(v_k) \log_2 \frac{1}{P(v_k)}$$

Somatório para cada valor v_k
Fazemos isso para todas as classes

Entropia

Shannon queria medir a **quantidade média de informação** (ou surpresa) ao de um evento aleatório X

$$H(V) = \sum_k P(v_k)\log_2 \frac{1}{P(v_k)} = \boxed{-\sum_k P(v_k)\log_2 P(v_k)}$$

Fórmula mais comum para representar entropia

Entropia

log(a/b) = log(a) - log(b)
= log(1) - log(P(v_k))
= - log(P(v_k))

$$H(V) = \sum_k P(v_k) \boxed{\log_2 \frac{1}{P(v_k)}} = \boxed{-\sum_k P(v_k) \log_2 P(v_k)}$$

Fórmula mais comum para representar entropia

Entropia

Shannon queria medir a **quantidade média de informação** (ou surpresa) ao de um evento aleatório X

$$H(V) = \sum_k P(v_k)\log_2 \frac{1}{P(v_k)} = \boxed{-\sum_k P(v_k)\log_2 P(v_k)}$$

Entropia é medida em bits

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

V = {v_1, v_2} = {*cara*, *coroa*} ← *valores possíveis (classes)*

P(*cara*) = ½ = 0,5

P(*coroa*) = ½ = 0,5

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

$$H(V) = -(\frac{1}{2}\log_2(\frac{1}{2}) + \frac{1}{2}\log_2(\frac{1}{2}))$$

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

$$H(V) = -\sum_{k} P(v_k)\log_2 P(v_k)$$

$$H(V) = -(\boxed{\frac{1}{2}\log_2(\frac{1}{2})} + \frac{1}{2}\log_2(\frac{1}{2}))$$

V_k = cara

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

$$H(V) = -(\frac{1}{2}\log_2(\frac{1}{2}) + \boxed{\frac{1}{2}\log_2(\frac{1}{2})})$$

V_k = coroa

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

$$H(V) = -(\frac{1}{2}\log_2(\frac{1}{2}) + \frac{1}{2}\log_2(\frac{1}{2}))$$

log(½) = -1

$$H(V) = -[\frac{1}{2}(-1) + \frac{1}{2}(-1)]$$

$$H(V) = 1$$

Entropia – Exemplo

Como calcular a entropia de uma **moeda honesta**?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

$$H(V) = -(\frac{1}{2}\log_2(\frac{1}{2}) + \frac{1}{2}\log_2(\frac{1}{2}))$$

log(½) = -1

$$H(V) = -[\frac{1}{2}(-1) + \frac{1}{2}(-1)]$$

$$\boxed{H(V) = 1}$$

Incerteza TOTAL
Saídas igualmente prováveis

Entropia – Exemplo

E se for uma **moeda viciada** que dá cara 99% do tempo?

Entropia – Exemplo

E se for uma **moeda viciada** que dá cara 99% do tempo?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

V = {v_1, v_2} = {*cara*, *coroa*} ← *valores possíveis (classes)*

P(*cara*) = 0,99

P(*coroa*) = 0,01

Entropia – Exemplo

E se for uma **moeda viciada** que dá cara 99% do tempo?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

$$H(V) = -(0,99\log_2(0,99) + 0,01\log_2(0,01))$$

$$H(V) \approx 0,08$$

Entropia – Exemplo

E se for uma **moeda viciada** que dá cara 99% do tempo?

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

$$H(V) = -(0,99\log_2(0,99) + 0,01\log_2(0,01))$$

$$\boxed{H(V) \approx 0,08}$$

Pouca incerteza
Vai sair "cara" quase sempre

Ok, mas e as Árvores de Decisão?

Ok, mas e as Árvores de Decisão?

função APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO(*exemplos, atributos, exemplos-pais*)
retorna uma árvore de decisão
  se exemplos é vazio **então retornar** VALOR-DA-MAIORIA (*exemplos_pais*)
  senão se todos os *exemplos* têm a mesma classificação **então retornar** a classificação
  senão se *atributos* é vazio **então retornar** VALOR-DA-MAIORIA(*exemplos*)
  senão
    $A \leftarrow \text{argmax}_{a \in \text{atributes}}$ IMPORTÂNCIA (a, *exemplos*)
    árvore ← uma nova árvore de decisão com teste de raiz A
    para cada valor v_k de A **faça**
      exs ← $\{e : e \in \textit{exemplos}$ **e** $e.A = v_k\}$
      subárvore ← APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO (*exs, atributos* — A, *exemplos*)
      adicionar uma ramificação à árvore com rótulo ($A = v_k$) e subárvore *subárvore*
    retornar *árvore*

Precisamos de uma **medida formal** para "muito importante" e "inútil"

Ok, mas e as Árvores de Decisão?

A função IMPORTÂNCIA deverá ter seu valor máximo quando o **atributo for perfeito**, e seu valor mínimo quando o atributo for **absolutamente inútil.**

- A equação de entropia foi adaptada para medir **incerteza de classificação** em algoritmos como ID3, C4.5 e seus sucessores.

Entropia no contexto de ADs

$$H(V) = -\sum_{k} P(v_k)\log_2 P(v_k)$$

- ***V*** é o conjunto de dados
- ***k*** representa cada classe (categoria) possível
- **$P(v_k)$** é a proporção de elementos em *V* pertencentes à classe v_k

Entropia no contexto de ADs

$$H(V) = -\sum_k P(v_k)\boxed{\log_2 P(v_k)}$$

A escolha da base 2 está relacionada com o fato de a entropia ser medida em “bits” na teoria da informação.

Entropia no contexto de ADs

$$H(V) = -\sum_k P(v_k)\boxed{\log_2 P(v_k)}$$

Importante: Se desejar que o ganho e a entropia esteja entre 0 e 1, a base do log precisa ser igual ao número de classes.

A escolha da base 2 está relacionada com o fato de a entropia ser medida em “bits” na teoria da informação.

Entropia no contexto de ADs

$$H(V) = -\sum_{k} P(v_k)\log_2 P(v_k)$$

Sinal negativo **garante que a entropia seja um valor positivo**, pois os logaritmos de probabilidades são números negativos (quando $P(v_k) < 1$)

Entropia no contexto de ADs

Se um conjunto de treinamento contiver *p* exemplos positivos e *n* exemplos negativos*, a entropia <u>do atributo meta/alvo</u> **em todo o conjunto** será:

$$H(V) = -\sum_{k} P(v_k)\log_2 P(v_k)$$

positivos *negativos*

$$H(alvo) = -[\boxed{\frac{p}{p+n}\, log_2(\frac{p}{p+n})} + \boxed{\frac{n}{p+n}\, log_2(\frac{n}{p+n})}]$$

*O total de elementos então é ***p+n***

Entropia no contexto de ADs

Se um conjunto de treinamento contiver *p* exemplos positivos e *n* exemplos negativos, a entropia do atributo meta/alvo **em todo o conjunto** será:

$$H(alvo) = -[\boxed{\frac{p}{p+n}} log_2(\frac{p}{p+n}) + \frac{n}{p+n} log_2(\frac{n}{p+n})]$$

Proporção de **exemplos positivos** no conjunto

Entropia no contexto de ADs

Se um conjunto de treinamento contiver *p* exemplos positivos e *n* exemplos negativos, a entropia do atributo meta/alvo **em todo o conjunto** será:

$$H(alvo) = -[\frac{p}{p+n} log_2(\frac{p}{p+n}) + \boxed{\frac{n}{p+n}} log_2(\frac{n}{p+n})]$$

Proporção de **exemplos negativos** no conjunto

Entropia no contexto de ADs

Se fizermos um teste em um **único atributo *A***, teremos apenas uma parte dessa informação

- Podemos descobrir quanto verificando a entropia após o teste desse atributo

Entropia no contexto de ADs

Qualquer atributo A com valores distintos *d* divide o conjunto de treinamento *E* em subconjuntos $E_1,\ldots,E_d$, de acordo com os valores de cada atributo

- É como se criasse *d* "pastas" de dados (uma para cada valor do atributo)
- Cada subconjunto (pasta) E_k tem p_k exemplos positivos *e* n_k exemplos negativos

Entropia no contexto de ADs

Não basta calcular a entropia de cada subconjunto E_k isoladamente, Precisamos saber o **impacto real** no conjunto todo:

$$H(atributo) = \sum_{k=1}^{d} \quad \textbf{???} \quad H(E_k)$$

Entropia no contexto de ADs

Não basta calcular a entropia de cada subconjunto E_k isoladamente, Precisamos saber o **impacto real** no conjunto todo:

$$H(atributo) = \sum_{k=1}^{d} \boxed{\frac{p_k + n_k}{p + n}} H(E_k)$$

Média ponderada pelo tamanho de cada subconjunto (tamanho relativo ao total)

Entropia no contexto de ADs

Não basta calcular a entropia de cada subconjunto E_k isoladamente, Precisamos saber o **impacto real** no conjunto todo:

Tamanho do **subconjunto E_k**

$$H(atributo) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} H(E_k)$$

Tamanho do **conjunto de dados**

Entropia no contexto de ADs

Não basta calcular a entropia de cada subconjunto E_k isoladamente, Precisamos saber o **impacto real** no conjunto todo:

$$H(atributo) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} \boxed{H(E_k)}$$

$$\boxed{H(E_k) = -[\frac{p_k}{p_k + n_k} \log_2(\frac{p_k}{p_k + n_k}) + \frac{n_k}{p_k + n_k} \log_2(\frac{n_k}{p_k + n_k})]}$$

Considerando **apenas** as instâncias do subconjunto em questão

Então...

Então basta usar a entropia para decidir o atributo mais importante?

Então...

Então basta usar a entropia para decidir o atributo mais importante? **Não!**

O algoritmo precisa saber <u>o quanto a entropia diminuiu</u> após a divisão.

Ganho de informação

O **ganho de informação** a partir do teste de atributo é a diferença entre a entropia original e a entropia do atributo

- Em outras palavras, é o quanto de incerteza aquele atributo consegue eliminar

Ganho de informação

O **ganho de informação** a partir do teste de atributo é a diferença entre a entropia original e a entropia do atributo

- Em outras palavras, é o quanto de incerteza aquele atributo consegue eliminar

$$ganho(atributo) = H(alvo) - H(atributo)$$

Finalmente...

função APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO(*exemplos, atributos, exemplos-pais*)
retorna uma árvore de decisão
  se exemplos é vazio **então retornar** VALOR-DA-MAIORIA (*exemplos_pais*)
  senão se todos os *exemplos* têm a mesma classificação **então retornar** a classificação
  senão se *atributos* é vazio **então retornar** VALOR-DA-MAIORIA(*exemplos*)
  senão
    $A \leftarrow \text{argmax}_{a \in \text{atributes}}$ IMPORTÂNCIA (a, *exemplos*)
    árvore ← uma nova árvore de decisão com teste de raiz A
    para cada valor v_k de A **faça**
      exs ← $\{e : e \in \textit{exemplos}$ **e** $e.A = v_k\}$
      subárvore ← APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO (*exs, atributos* — A, *exemplos*)
      adicionar uma ramificação à árvore com rótulo ($A = v_k$) e subárvore *subárvore*
    retornar *árvore*

Finalmente...

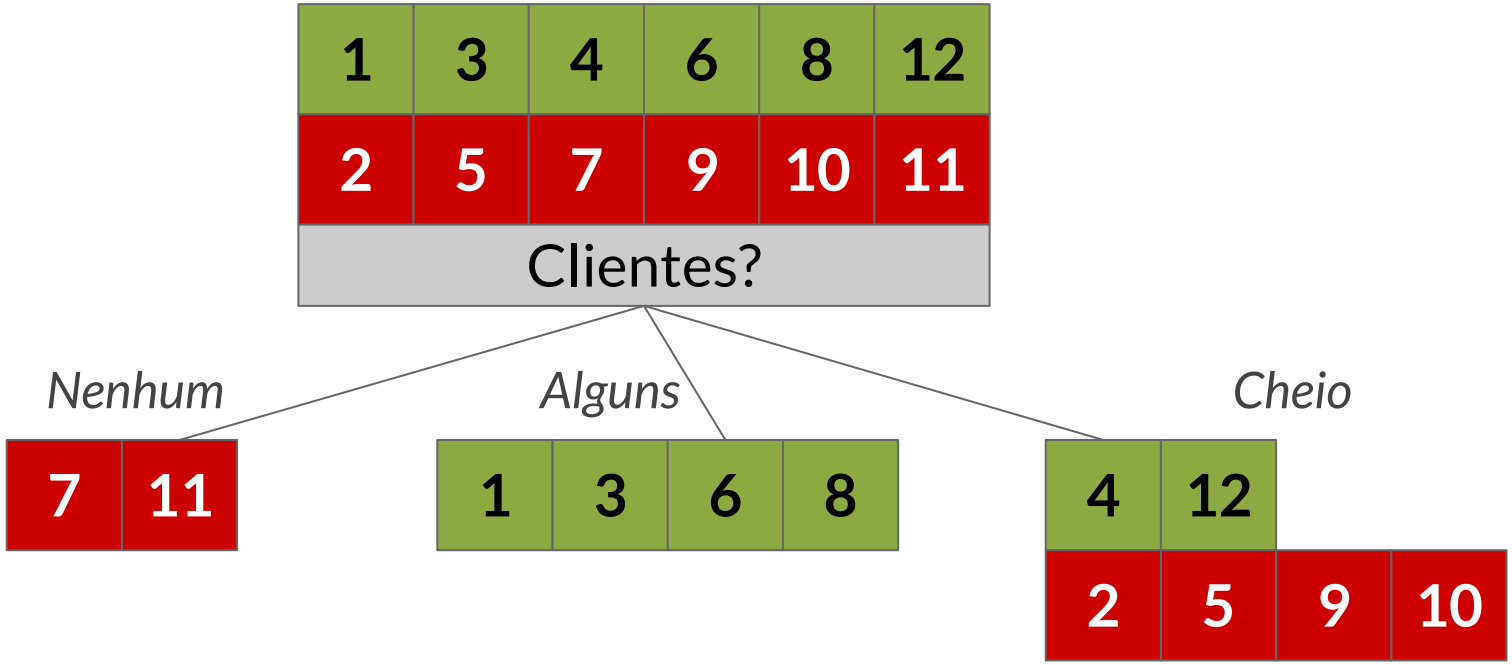
função APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO(*exemplos, atributos, exemplos-pais*)
retorna uma árvore de decisão
  se exemplos é vazio **então retornar** VALOR-DA-MAIORIA (*exemplos_pais*)
  senão se todos os *exemplos* têm a mesma classificação **então retornar** a classificação
  senão se *atributos* é vazio **então retornar** VALOR-DA-MAIORIA(*exemplos*)
  senão
    $A \leftarrow \text{argmax}_{a \in atributes}$ IMPORTÂNCIA (a, *exemplos*)
    árvore ← uma nova árvore de decisão com teste de raiz A
    para cada valor v_k de A **faça**
      exs ← $\{e : e \in exemplos$ **e** $e.A = v_k\}$
      subárvore ← APRENDIZAGEM-EM-ÁRVORE-DE-DECISÃO (*exs, atributos* — A, *exemplos*)
      adicionar uma ramificação à árvore com rótulo ($A = v_k$) e subárvore *subárvore*
    retornar *árvore*

A heurística usada na função IMPORTÂNCIA é escolher o atributo com **maior ganho**

Exemplo – Esperar uma mesa?

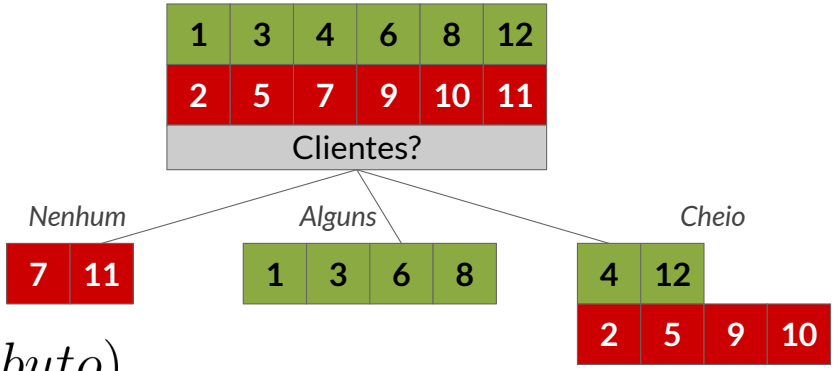
Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?



Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?

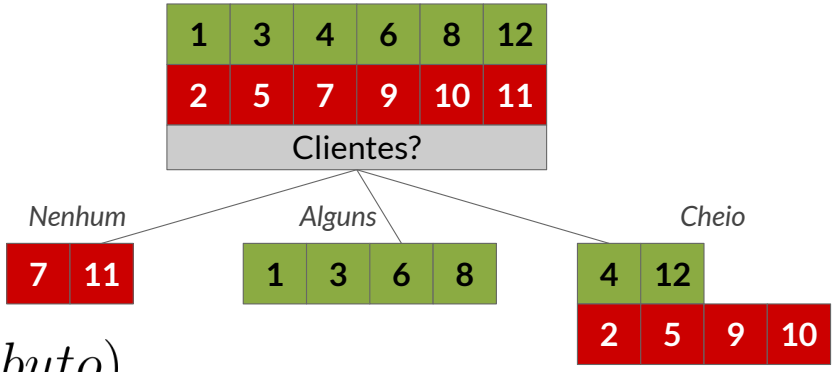


$$ganho(atributo) = H(alvo) - H(atributo)$$
$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

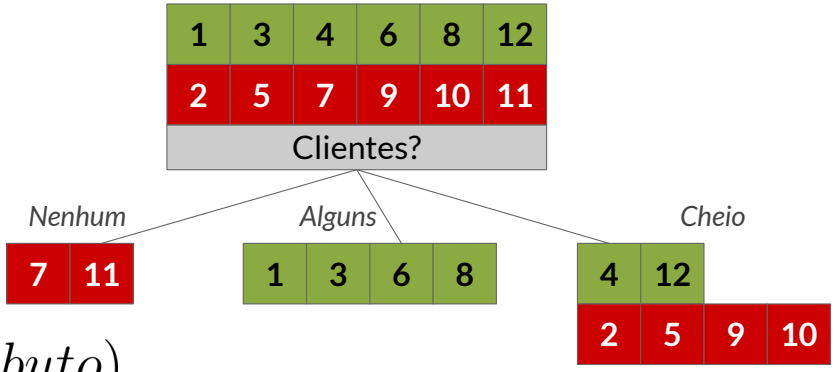
$$H(VaiEsperar) = -[\overbrace{\frac{1}{2}\log_2(\frac{1}{2})}^{\text{"Sim"}} + \overbrace{\frac{1}{2}\log_2(\frac{1}{2})}^{\text{"Não"}}] = 1$$

A proporção de exemplos nas duas classes ("**Sim**" e "**Não**") é 6/12 = 1/2

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

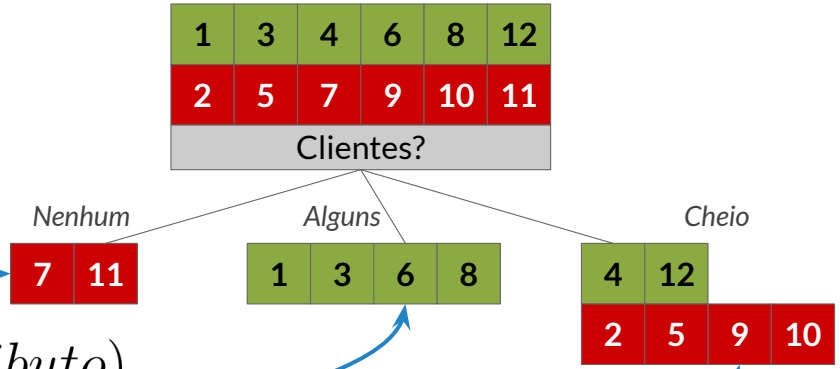
$$ganho(Clientes) = 1 - [\frac{2}{12}H(E_{Nenhum}) + \frac{4}{12}H(E_{Alguns}) + \frac{6}{12}H(E_{Cheio})]$$

$$H(atributo) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} H(E_k)$$

Lembre-se: Precisamos ponderar pelo tamanho relativo dos subconjuntos!

Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

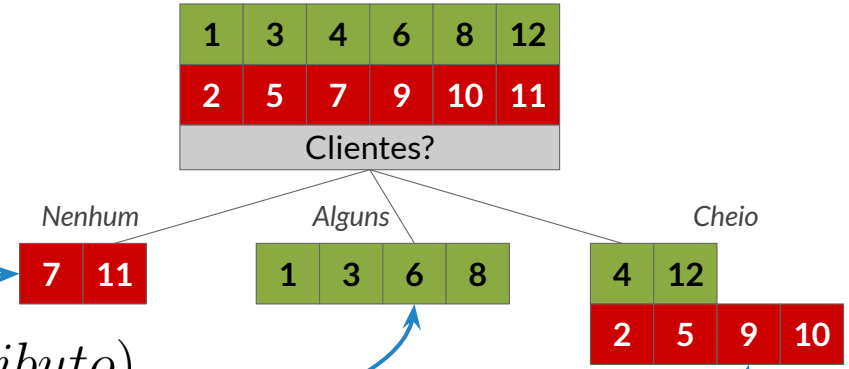
$$ganho(Clientes) = 1 - [\boxed{\frac{2}{12}} H(E_{Nenhum}) + \boxed{\frac{4}{12}} H(E_{Alguns}) + \boxed{\frac{6}{12}} H(E_{Cheio})]$$

Proporção de elementos nos subconjuntos
"Nenhum", "Alguns" e **"Cheio"**

$$H(atributo) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} H(E_k)$$

Exemplo – Esperar uma mesa?

Qual o ganho do atributo **"Clientes"**?



$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

$$ganho(Clientes) = 1 - [\frac{2}{12}H(E_{Nenhum}) + \frac{4}{12}H(E_{Alguns}) + \frac{6}{12}H(E_{Cheio})]$$

p(Sim) = 0
p(Não) = 1

p(Sim) = 1
p(Não) = 0

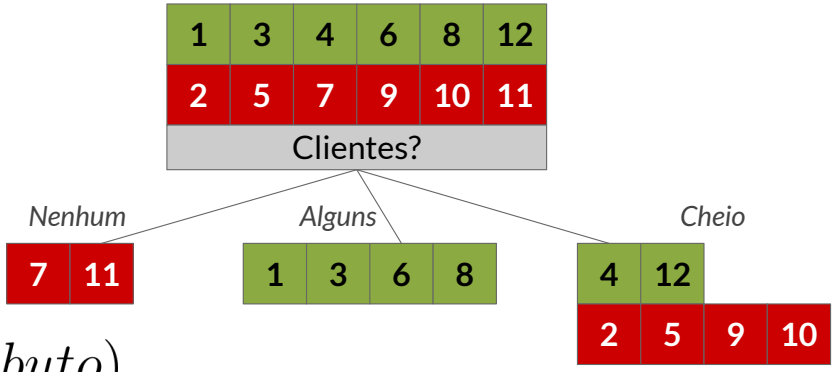
p(Sim) = 2/6
p(Não) = 4/6

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

Probabilidades a serem inseridas no cálculo da entropia para os subconjuntos **"Nenhum"**, **"Alguns"** e **"Cheio"**

Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

$$ganho(Clientes) = 1 - [\frac{2}{12}H(E_{Nenhum}) + \frac{4}{12}H(E_{Alguns}) + \frac{6}{12}H(E_{Cheio})]$$

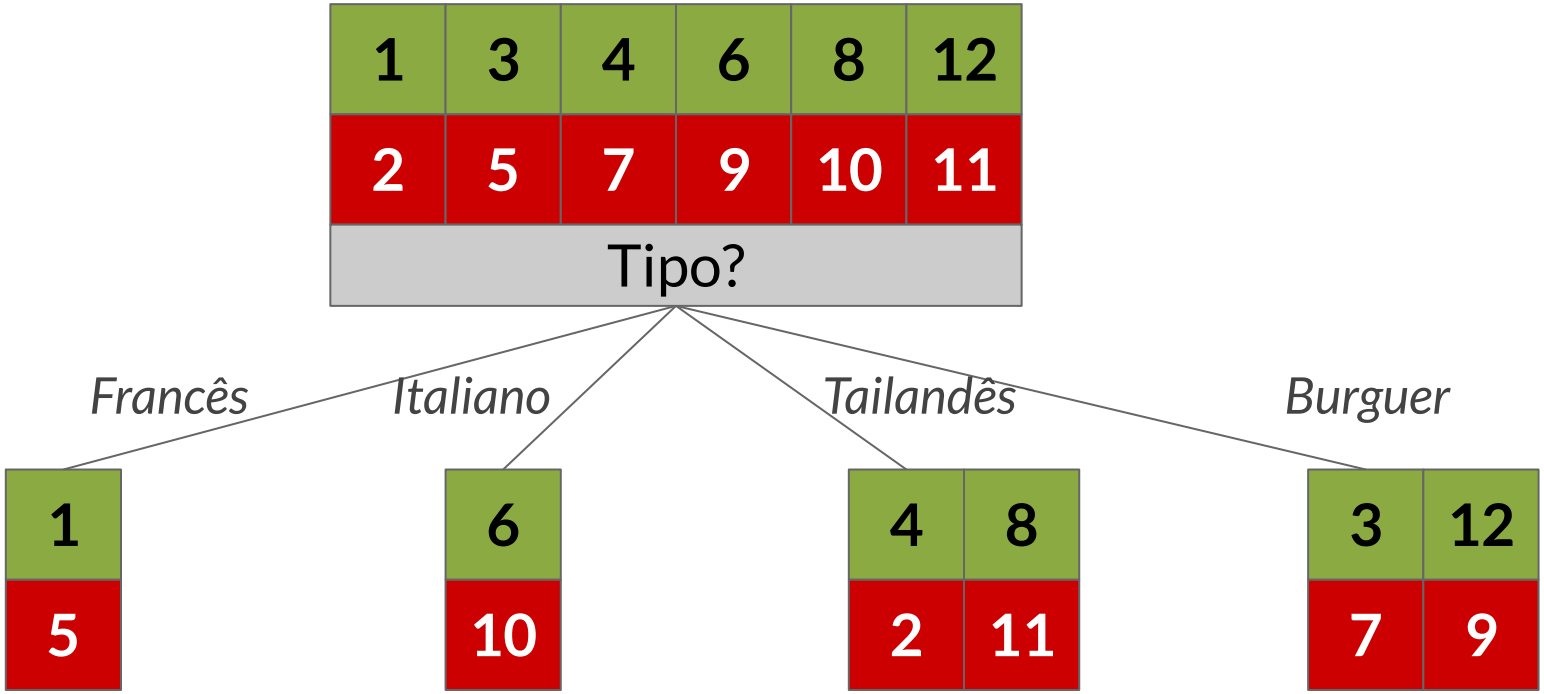
$$ganho(Clientes) \approx 0,541$$

Exemplo – Esperar uma mesa?

Qual o ganho do atributo “**Tipo**”?

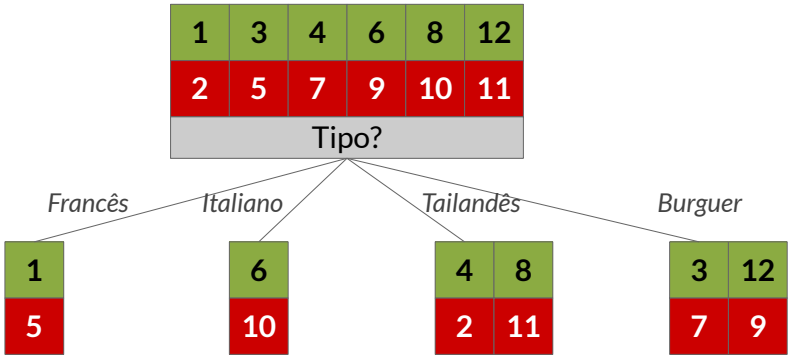
Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Tipo**"?



Exemplo – Esperar uma mesa?

Qual o ganho do atributo **"Tipo"**?

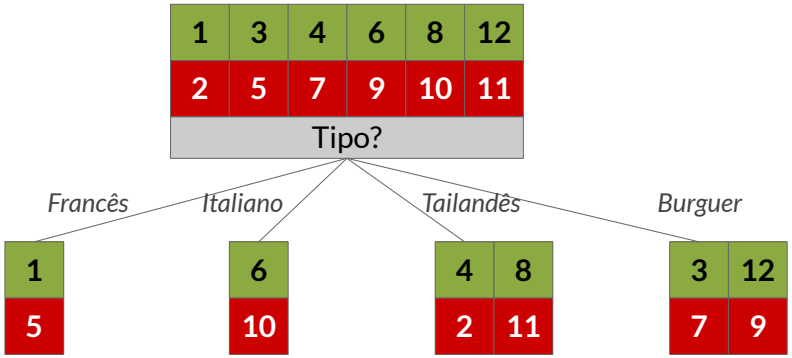


$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Tipo) = 1 - [\frac{2}{12}H(E_{Frances}) + \frac{2}{12}H(E_{Italiano}) + \frac{4}{12}H(E_{Tailandes}) + \frac{4}{12}H(E_{Burguer})]$$

Exemplo – Esperar uma mesa?

Qual o ganho do atributo **"Tipo"**?



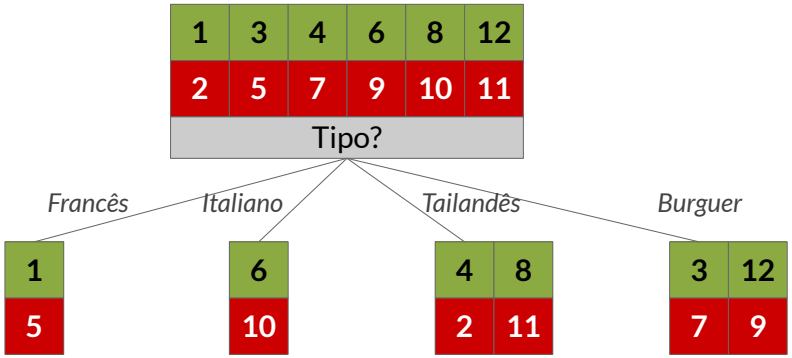
$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Tipo) = \boxed{1} - [\frac{2}{12}H(E_{Frances}) + \frac{2}{12}H(E_{Italiano}) + \frac{4}{12}H(E_{Tailandes}) + \frac{4}{12}H(E_{Burguer})]$$

Já sabemos que a entropia do conjunto total é 1

Exemplo – Esperar uma mesa?

Qual o ganho do atributo **"Tipo"**?



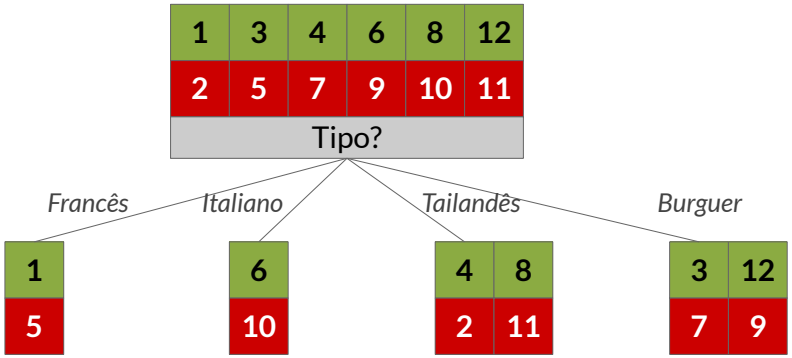
$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Tipo) = 1 - [\boxed{\frac{2}{12}} H(E_{Frances}) + \boxed{\frac{2}{12}} H(E_{Italiano}) + \boxed{\frac{4}{12}} H(E_{Tailandes}) + \boxed{\frac{4}{12}} H(E_{Burguer})]$$

Proporção de elementos nos subconjuntos
"Francês", "Italiano", "Tailandês" e **"Burguer"**

Exemplo – Esperar uma mesa?

Qual o ganho do atributo **"Tipo"**?



$$ganho(atributo) = H(alvo) - H(atributo)$$

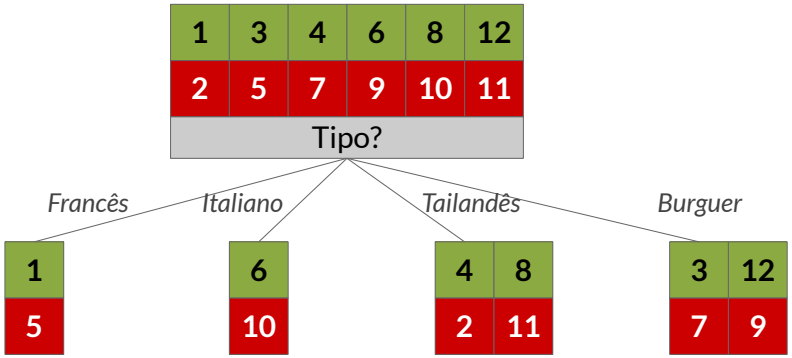
$$ganho(Tipo) = 1 - [\frac{2}{12}\boxed{H(E_{Frances})} + \frac{2}{12}\boxed{H(E_{Italiano})} + \frac{4}{12}\boxed{H(E_{Tailandes})} + \frac{4}{12}\boxed{H(E_{Burguer})}]$$

(1/2, 1/2) (1/2, 1/2) (1/2, 1/2) (1/2, 1/2)

Probabilidades a serem inseridas no cálculo da entropia para os subconjuntos **"Francês", "Italiano", "Tailandês"** e **"Burguer"**

Exemplo – Esperar uma mesa?

Qual o ganho do atributo **"Tipo"**?



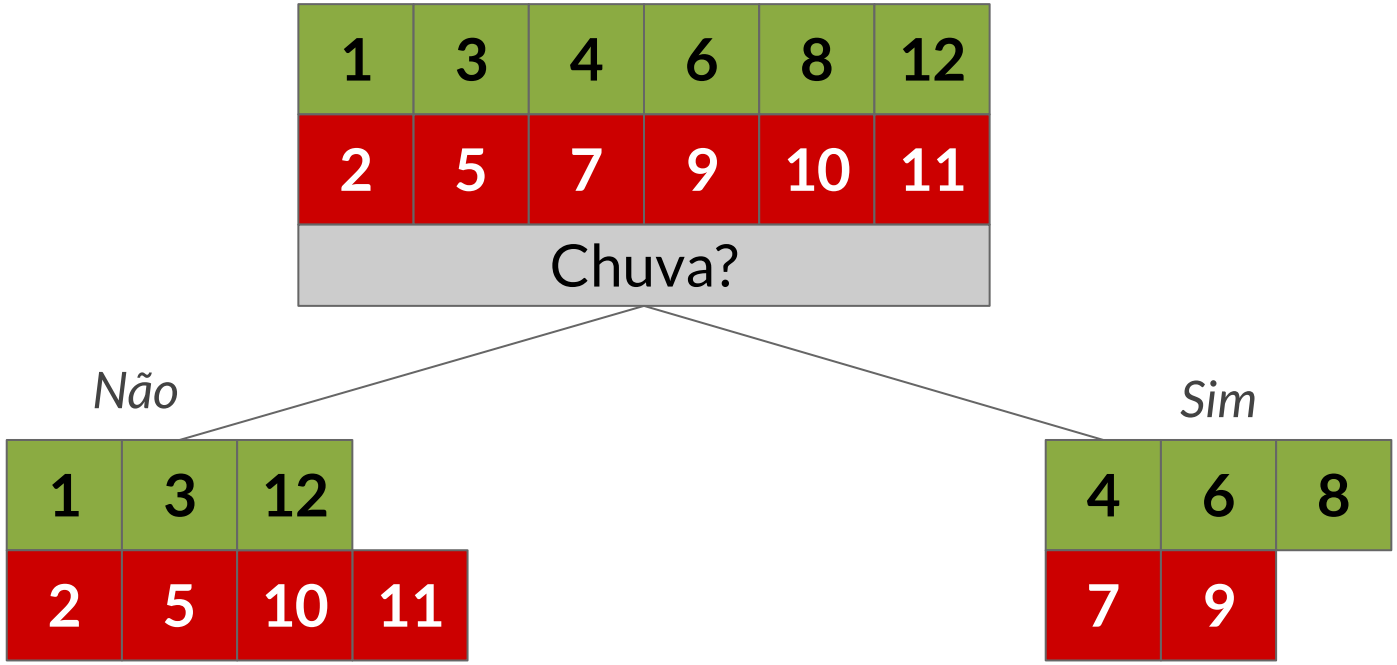
$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Tipo) = 1 - [\frac{2}{12}H(E_{Frances}) + \frac{2}{12}H(E_{Italiano}) + \frac{4}{12}H(E_{Tailandes}) + \frac{4}{12}H(E_{Burguer})]$$

$$ganho(Tipo) = 0$$

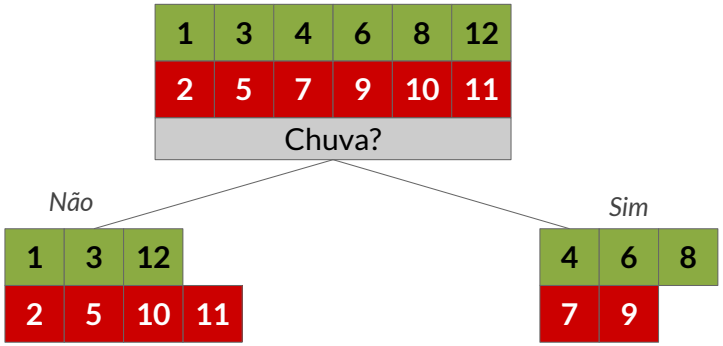
Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Chuva**"?



Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Chuva**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$

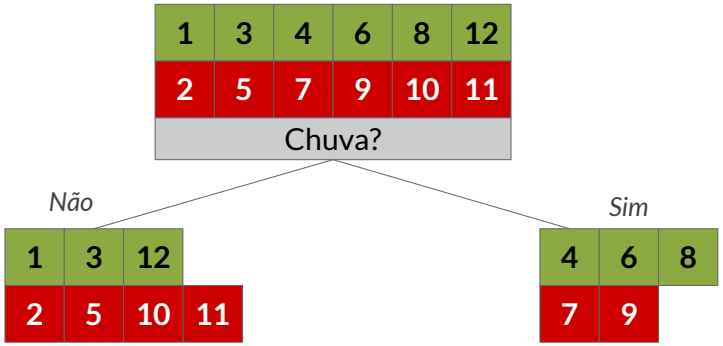
$$ganho(Chuva) = 1 - [\frac{5}{12}H(E_{Sim}) + \frac{7}{12}H(E_{Nao})]$$

(3/5, 2/5) (3/7, 4/7)

$$ganho(Chuva) \approx 0,021$$

Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Chuva**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$

$$ganho(Chuva) = 1 - [\frac{5}{12}H(E_{Sim}) + \frac{7}{12}H(E_{Nao})]$$

(3/5, 2/5) (3/7, 4/7)

$$ganho(Chuva) \approx 0,021$$

É melhor ou pior que os atributos "Tipo" e "Clientes"?

Escolha de testes de atributos

Como medir a habilidade de um dado atributo discriminar as classes?

Existem várias formas, mas todas concordam em:

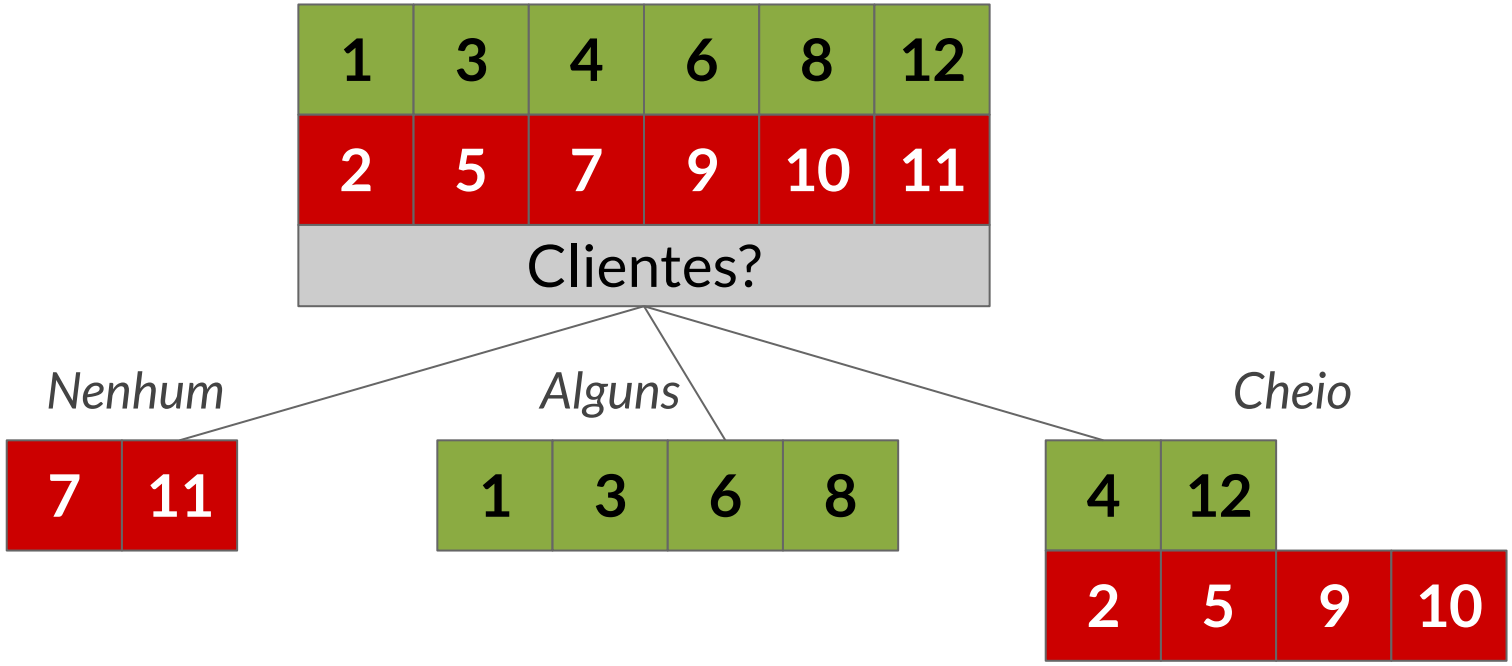
- Uma divisão que mantêm as proporções de classes em todas as partições é **inútil**
- Uma divisão onde em cada partição todos os exemplos são da mesma classe tem **utilidade máxima**

Em resumo: o algoritmo

1. Escolher um atributo
2. Estender a árvore **adicionando um ramo para cada valor do atributo**
3. Passar os exemplos para as folhas (de acordo com o valor do atributo)
4. Para cada folha
   a. Se todos os exemplos são da mesma classe, associar esta classe à folha 2
   b. Senão, repetir os passos 1 a 4

Exemplo – Esperar uma mesa?

Considerando que o atributo **Cliente** estará na raiz da árvore (**maior ganho**), que atributos estarão no segundo nível da árvore?

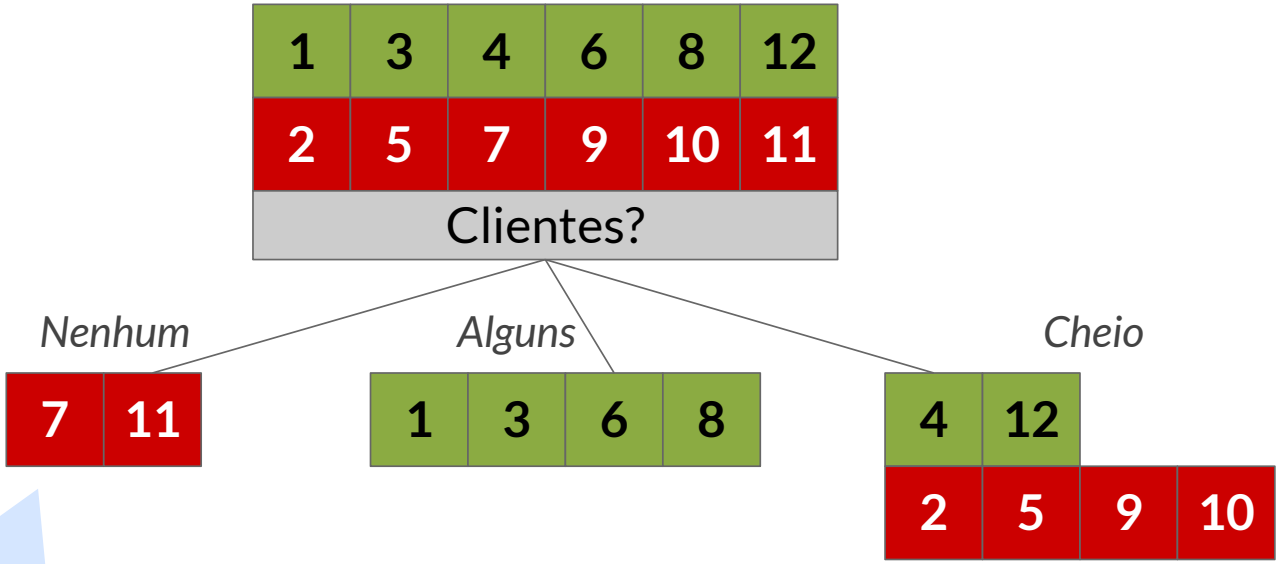


Exemplo – Esperar uma mesa?

Depois que o primeiro teste de atributo separar os exemplos, cada resultado é um **novo problema de aprendizagem de árvore de decisão** em si, com menos exemplos e um atributo a menos.

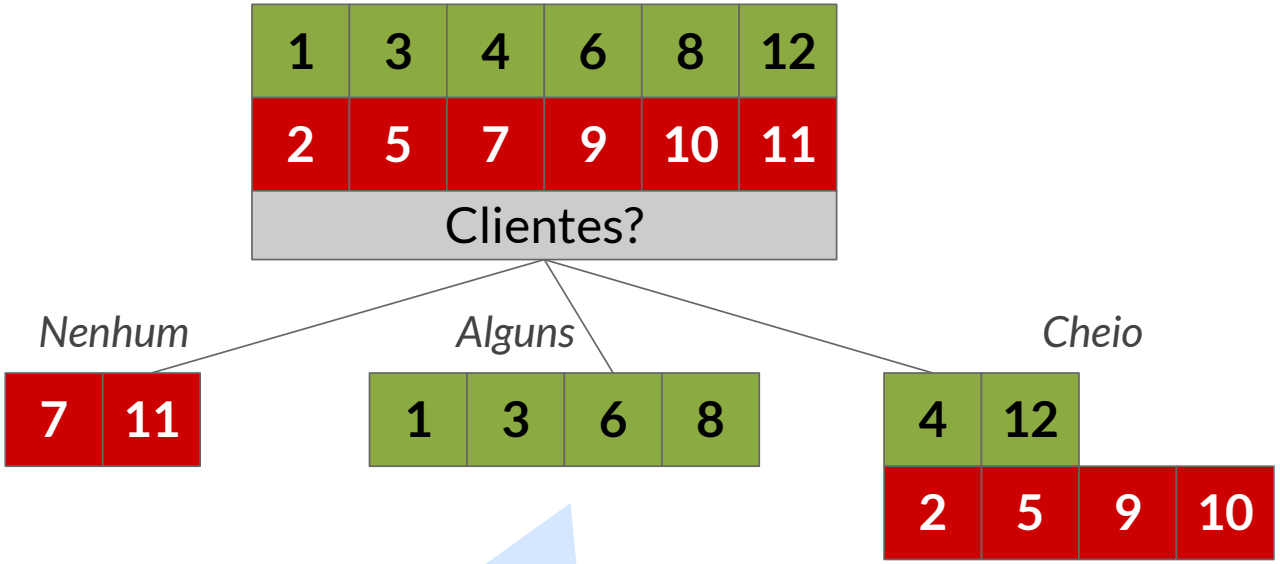
Dividir para conquistar :-D

Exemplo – Esperar uma mesa?



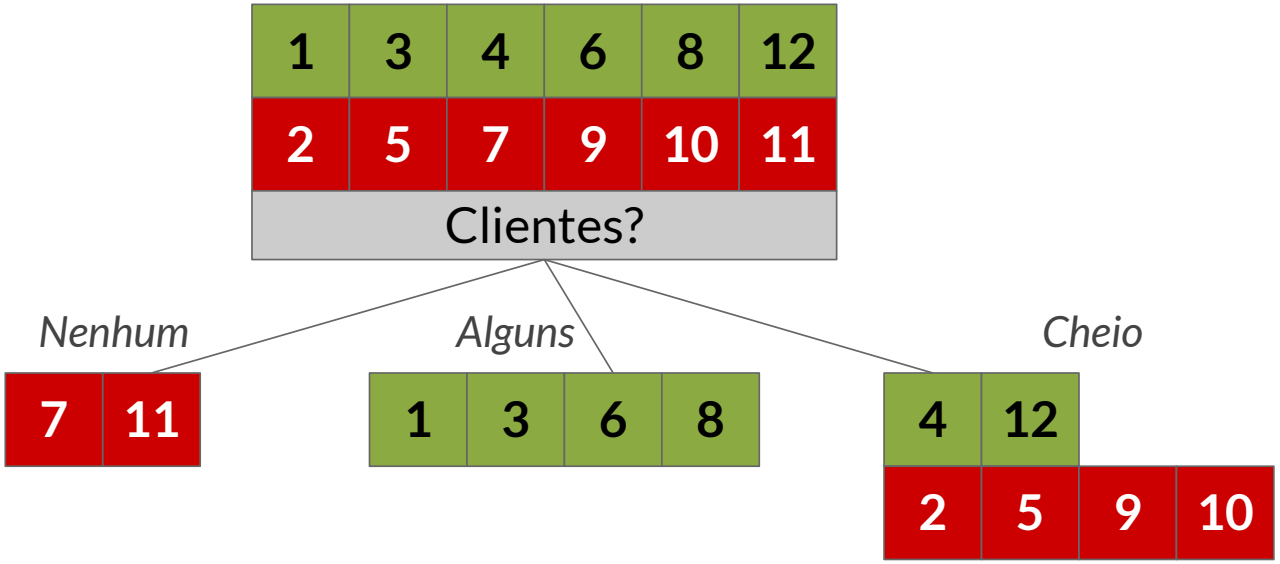
Quando **Cliente=Nenhum**, todas as instâncias são iguais a **Não**

Exemplo – Esperar uma mesa?



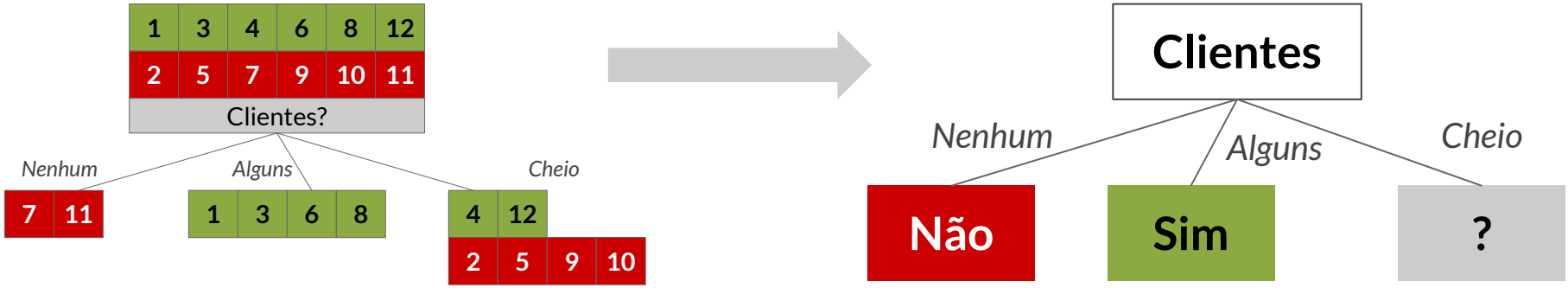
Quando **Cliente=Alguns**, todas as instâncias são iguais a **Sim**

Exemplo – Esperar uma mesa?

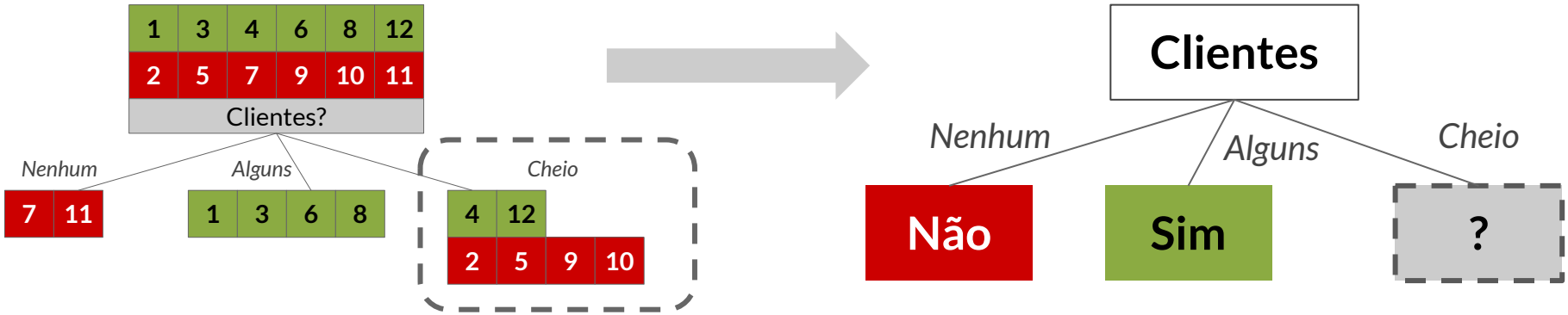


Quando **Cliente=Cheio**, ainda temos instâncias das duas classes, então precisamos de <u>outro atributo para separar os exemplos</u>

Exemplo – Esperar uma mesa?

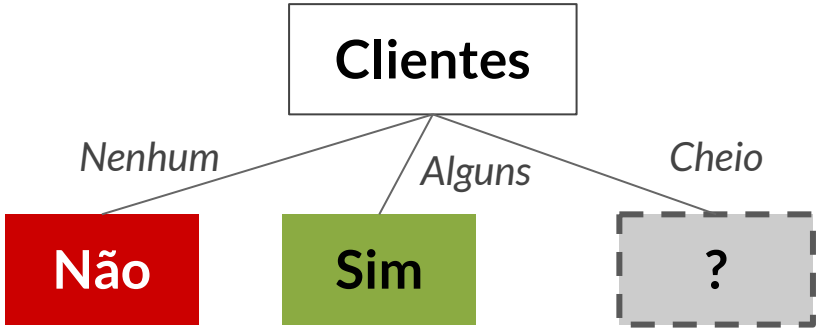


Exemplo – Esperar uma mesa?



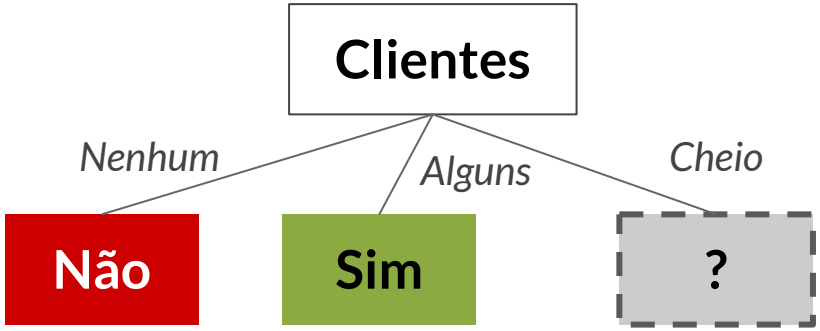
O processo de geração da árvore continuará **apenas com estes seis exemplos** e precisamos descobrir um <u>novo atributo</u> para separá-los

Exemplo – Esperar uma mesa?



Exemplo	Alt.	Bar	Sex/Sab	Fome	Clientes	Pr.	Chuva	Res.	Tipo	Estim.	VaiEsperar
x2	Sim	Não	Não	Sim	Cheio	$	Não	Não	Tailandês	30-60	Não
x4	Sim	Não	Sim	Sim	Cheio	$	Sim	Não	Tailandês	10-30	Sim
x5	Sim	Não	Sim	Não	Cheio	$$$	Não	Sim	Francês	>60	Não
x9	Não	Sim	Sim	Não	Cheio	$	Sim	Não	Burguer	>60	Não
x10	Sim	Sim	Sim	Sim	Cheio	$$$	Não	Sim	Italiano	10-30	Não
x12	Sim	Sim	Sim	Sim	Cheio	$	Não	Não	Burguer	30-60	Sim

Exemplo – Esperar uma mesa?



Qual será o novo atributo?
Basta executar o algoritmo de novo!

Exemplo	Alt.	Bar	Sex/Sab	Fome	Clientes	Pr.	Chuva	Res.	Tipo	Estim.	VaiEsperar
x2	Sim	Não	Não	Sim	Cheio	$	Não	Não	Tailandês	30-60	Não
x4	Sim	Não	Sim	Sim	Cheio	$	Sim	Não	Tailandês	10-30	Sim
x5	Sim	Não	Sim	Não	Cheio	$$$	Não	Sim	Francês	>60	Não
x9	Não	Sim	Sim	Não	Cheio	$	Sim	Não	Burguer	>60	Não
x10	Sim	Sim	Sim	Sim	Cheio	$$$	Não	Sim	Italiano	10-30	Não
x12	Sim	Sim	Sim	Sim	Cheio	$	Não	Não	Burguer	30-60	Sim

2.

Algoritmo ID3

Como funciona?

Algoritmo ID3

ID3 → Iterative Dichotomiser 3

Proposto por J. R. Quinlan na Universidade de Sydney (Austrália) nos anos 1970

Induction of Decision Trees

J.R. QUINLAN (munnari!nswitgould.oz!quinlan@seismo.css.gov)
Centre for Advanced Computing Sciences, New South Wales Institute of Technology, Sydney 2007, Australia

(Received August 1, 1985)

Key words: classification, induction, decision trees, information theory, knowledge acquisition, expert systems

https://link.springer.com/article/10.1007/BF00116251

Algoritmo ID3

ID3 → Iterative Dichotomiser 3

É considerado o algoritmo de AD mais clássico

Constrói a árvore de decisão usando **entropia** e **ganho de informação**

É recursivo e baseado em busca gulosa, procurando o conjunto de atributos que melhor dividem as amostras (exemplos), gerando sub-árvores

Algoritmo ID3

ID3(*Exemplos*, *Atributo-objetivo*, *Atributos*)

// **ID3** retorna uma árvore de decisão que classifica corretamente os ***Exemplos*** determinados

// ***Exemplos*** são os exemplos de treinamento.

// ***Atributo-objetivo*** é o atributo cujo valor deve ser predito pela árvore.

// ***Atributos*** são uma lista de outros atributos que podem ser testados pela árvore de decisão.

Início

Crie um nodo ***Raiz*** para a árvore

Se todos os ***Exemplos*** são positivos

Então retorna a ***Raiz*** da árvore com o rótulo = **sim**

Se todos os ***Exemplos*** são negativos

Então retorna a ***Raiz*** da árvore com o rótulo = **não**

Se ***Atributos*** for vazio

Então retorna a ***Raiz*** da árvore com o rótulo = valor mais comum do ***Atributo-objetivo*** em ***Exemplos***

Senão

A ← um atributo de ***Atributos*** que melhor classifica ***Exemplos*** (atributo de decisão)

Raiz ← **A** (rótulo = atributo de decisão **A**)

Para cada possível valor v_i de **A** faça

Acrescenta um novo arco abaixo da ***Raiz***, correspondendo à resposta **A** = v_i

Seja $Exemplos_{vi}$ o subconjunto de ***Exemplos*** que têm valor v_i para **A**

Se $Exemplos_{vi}$ for vazio

Então acrescenta na extremidade do arco um nodo folha

com rótulo = valor mais comum do ***Atributo-objetivo*** em ***Exemplos***

Senão acrescenta na extremidade do arco a sub árvore

ID3($Exemplos_{vi}$, *Atributo-objetivo*, *Atributos* − {**A**})

Retorna ***Raiz*** (aponta para a árvore)

Fim

Algoritmo ID3 - Limitações

- **Não lida com atributos contínuos** (que devem ser discretizados)
- **Não lida com valores desconhecidos**, ou seja, todos os exemplos do conjunto de treinamento deve ter valores conhecidos para todos os seus atributos
- O algoritmo também **não lida com nenhum mecanismo de poda**, o que poderia amenizar árvores mais complexas

3.

Algoritmo C4.5

Como funciona?

Algoritmo C4.5

Proposto por J. R. Quinlan em 1993 como uma **extensão do algoritmo ID3.**

A ideia era lidar com as limitações existentes no algoritmo anterior.

Algoritmo C4.5 – Melhorias

Lidar com atributos contínuos e discretos

Para atributos numéricos, o algoritmo C4.5 **define um valor de limiar** e divide os dados em dois grupos:

- Valor do atributo <= limiar (ponto de referência)
- Valor do atributo > limiar (ponto de referência)

Algoritmo C4.5 – Melhorias

Escolha do ponto de referência

- Ordenar os exemplos por **ordem crescente** dos valores do atributo numérico
- Na prática, costuma-se testar os pontos médios entre valores adjacentes como candidatos a limiar
- Calcula-se o ganho de informação (ou razão de ganho) para cada limiar
- Fayyard e Irani (1993) mostram que de todos os possíveis pontos de referência, **aqueles que maximizam o ganho de informação separam dois exemplos de classes diferentes**

Algoritmo C4.5 – Melhorias

Poda de árvores após a criação

O algoritmo C4.5 retrocede pela árvore quando ela é criada e tenta **remover ramificações que não ajudam no processo de decisão,** substituindo-os por nós folha.

Algoritmo C4.5 – Melhorias

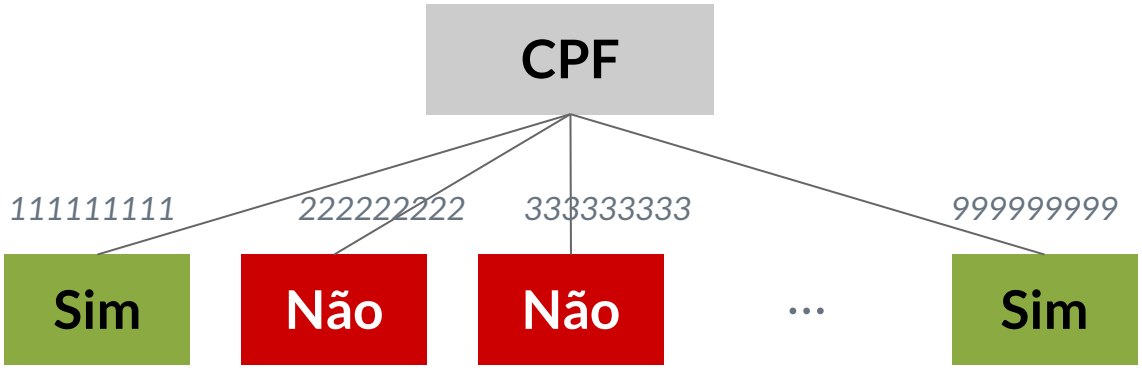
Implementa a <u>Razão de Ganho</u> (*gain ratio)* no lugar do ganho de informação tradicional

O ID3 tinha um problema: ele favorecia atributos com muitos valores diferentes (como um CPF ou ID), o que gerava árvores inúteis.

A Razão de Ganho foi introduzida para **corrigir o viés do Ganho de Informação**: ela normaliza o ganho de informação ao considerar o número de valores distintos de um atributo.

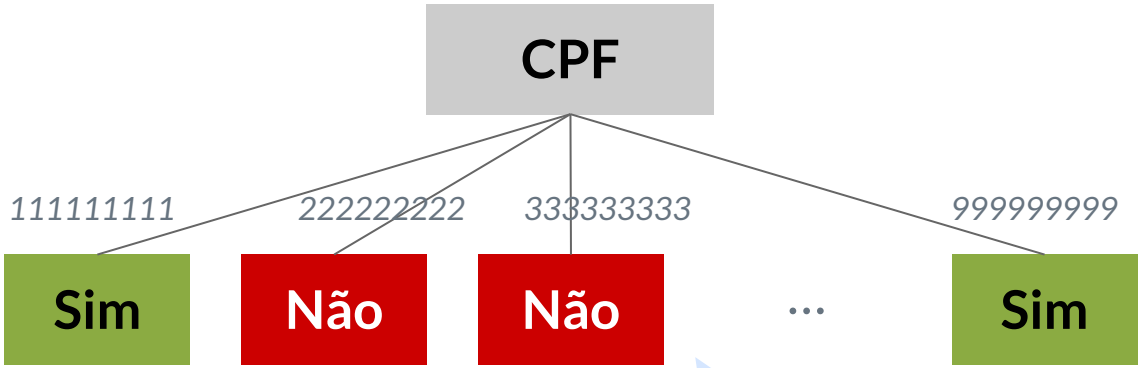
Razão de Ganho (*gain ratio*)

Exemplo extremo: Imagine usar o CPF (único para cada pessoa) como um atributo na raiz da árvore.



Razão de Ganho (*gain ratio*)

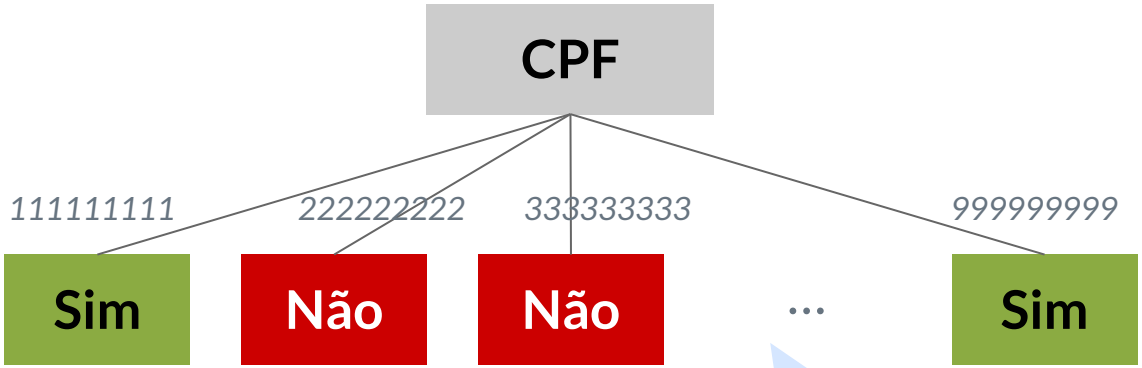
Exemplo extremo: Imagine usar o CPF (único para cada pessoa) como um atributo na raiz da árvore.



Aqui todo subconjunto (de um elemento) contém exemplos de uma mesma classe, ou seja, **o ganho de informação do atributo será máximo (entropia de cada subconjunto é 0)**

Razão de Ganho (*gain ratio*)

Exemplo extremo: Imagine usar o CPF (único para cada pessoa) como um atributo na raiz da árvore.



Não é bem um teste muito útil...
Overfitting do modelo (ele "decorou" as instâncias)

Razão de Ganho (*gain ratio*)

O C4.5 resolve isso **normalizando o ganho**. Ele divide o ganho de informação por uma métrica chamada **SplitInfo** (Informação de Divisão).

A ideia aqui é penalizar o atributo que **divide em vários subconjuntos <u>muito pequenos</u>**

Como ele faz isso?

Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{SplitInfo(atributo)}$$

Razão de Ganho (*gain ratio*)

$$\boxed{ganho(atributo) = H(alvo) - H(atributo)}$$

$$GainRatio(atributo) = \frac{Ganho(atributo)}{SplitInfo(atributo)}$$

Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{\boxed{SplitInfo(atributo)}}$$

Novidade! É a informação potencial gerada pela partição em *d* subconjuntos

Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{\boxed{SplitInfo(atributo)}}$$

$$SplitInfo(atributo) = -\sum_i \frac{|S_i|}{|S|} \, log_2 \left(\frac{|S_i|}{|S|} \right)$$

É basicamente a entropia da **distribuição dos tamanhos dos subconjuntos.**

Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{\boxed{SplitInfo(atributo)}}$$

$$SplitInfo(atributo) = -\sum_i \boxed{\frac{|S_i|}{|S|}} log_2 \left(\boxed{\frac{|S_i|}{|S|}} \right)$$

É a o tamanho relativo do subconjunto S_i em relação ao conjunto total S

Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{\boxed{SplitInfo(atributo)}}$$

$$SplitInfo(atributo) = -\sum_i \frac{|S_i|}{|S|} \, log_2 \left(\frac{|S_i|}{|S|} \right)$$

Consideramos todos os subconjuntos S_i que o atributo gera

Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{\boxed{SplitInfo(atributo)}}$$

$$SplitInfo(atributo) = -\sum_i \frac{|S_i|}{|S|} \, log_2 \left(\frac{|S_i|}{|S|} \right)$$

SplitInfo mede **o quanto o atributo fragmentou os dados**.

Se o atributo gera:

- **Muitos subconjuntos pequenos** → SplitInfo alto
- **Poucos subconjuntos grandes** → SplitInfo menor

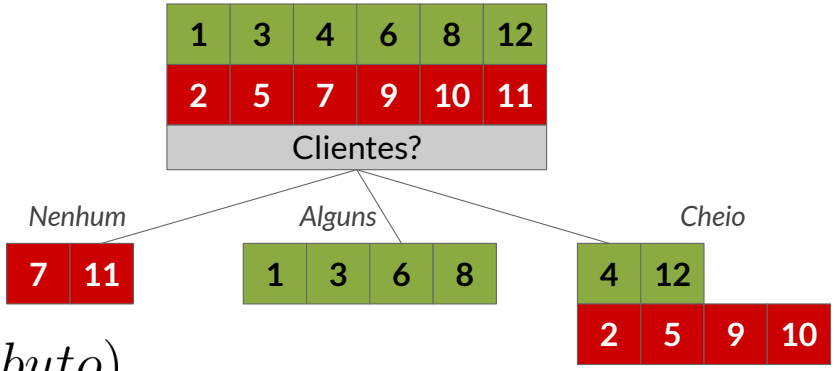
Razão de Ganho (*gain ratio*)

$$GainRatio(atributo) = \frac{Ganho(atributo)}{SplitInfo(atributo)}$$

A **razão de ganho** expressa a proporção de informação gerada pela partição que é útil, ou seja, que aparenta ser útil para a classificação

Exemplo – Esperar uma mesa?

Qual o ganho do atributo "**Clientes**"?



$$ganho(atributo) = H(alvo) - H(atributo)$$
$$ganho(Clientes) = H(VaiEsperar) - H(Clientes)$$

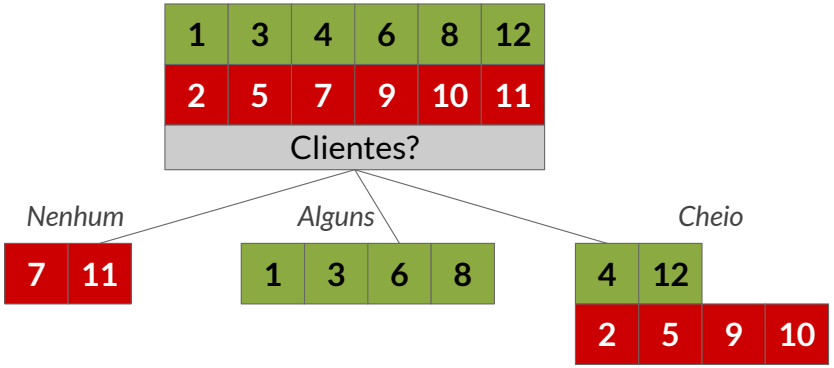
$$ganho(Clientes) = 1 - [\frac{2}{12}H(E_{Nenhum}) + \frac{4}{12}H(E_{Alguns}) + \frac{6}{12}H(E_{Cheio})]$$

$$ganho(Clientes) \approx 0,541$$

Exemplo – Esperar uma mesa?

Qual a **razão de ganho** do atributo "**Clientes**"?

$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$

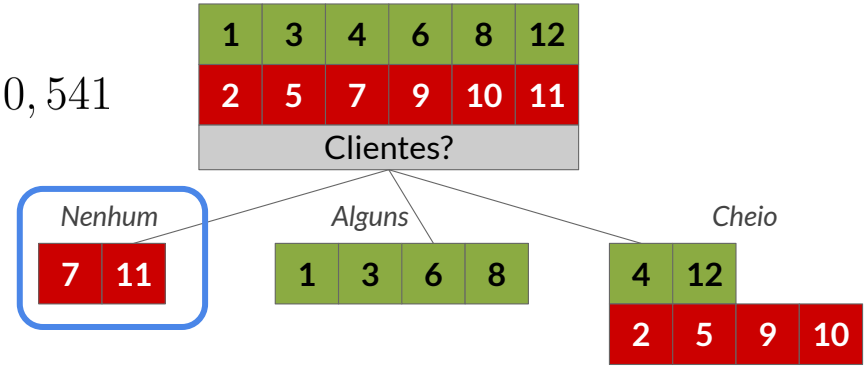


Exemplo – Esperar uma mesa?

Qual a **razão de ganho** do atributo **"Clientes"**?

$$ganho(Clientes) \approx 0,541$$

$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$



$$SplitInfo(Clientes) = -\sum_i \frac{|S_i|}{|S|} log_2\left(\frac{|S_i|}{|S|}\right)$$

$$= -\left[\boxed{\frac{2}{12} log_2\left(\frac{2}{12}\right)} + \frac{4}{12} log_2\left(\frac{4}{12}\right) + \frac{6}{12} log_2\left(\frac{6}{12}\right)\right]$$

Exemplo – Esperar uma mesa?

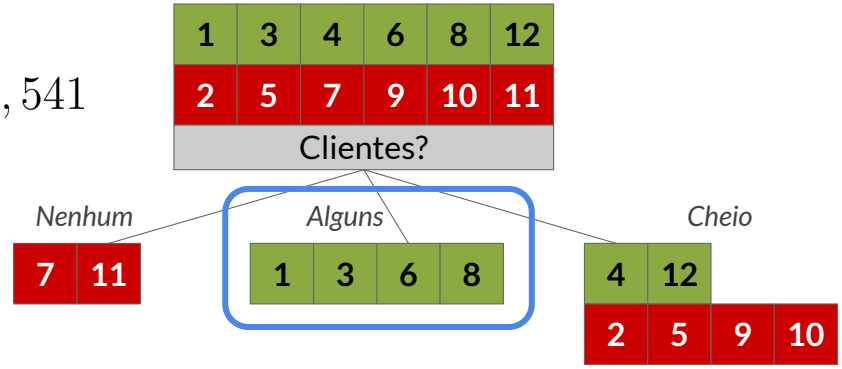
Qual a **razão de ganho** do atributo "**Clientes**"?

$$ganho(Clientes) \approx 0,541$$

$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$

$$SplitInfo(Clientes) = -\sum_i \frac{|S_i|}{|S|} log_2\left(\frac{|S_i|}{|S|}\right)$$

$$= -\left[\frac{2}{12} log_2\left(\frac{2}{12}\right) + \boxed{\frac{4}{12} log_2\left(\frac{4}{12}\right)} + \frac{6}{12} log_2\left(\frac{6}{12}\right)\right]$$



Exemplo – Esperar uma mesa?

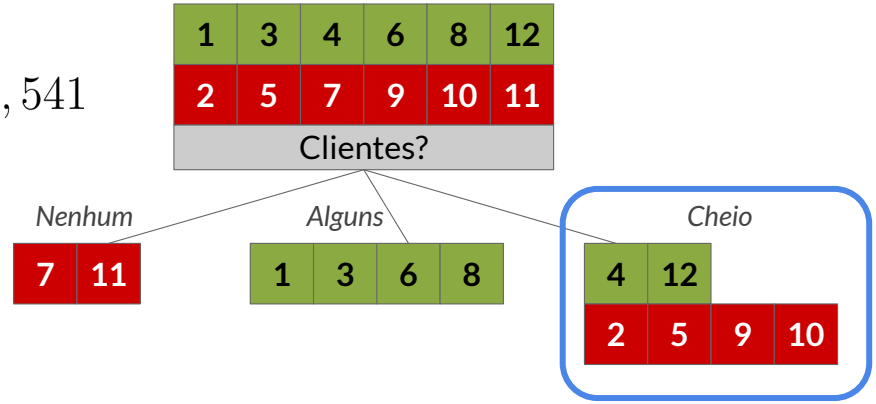
Qual a **razão de ganho** do atributo "**Clientes**"?

$$ganho(Clientes) \approx 0,541$$

$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$

$$SplitInfo(Clientes) = -\sum_i \frac{|S_i|}{|S|} log_2\left(\frac{|S_i|}{|S|}\right)$$

$$= -\left[\frac{2}{12} log_2\left(\frac{2}{12}\right) + \frac{4}{12} log_2\left(\frac{4}{12}\right) + \boxed{\frac{6}{12} log_2\left(\frac{6}{12}\right)}\right]$$

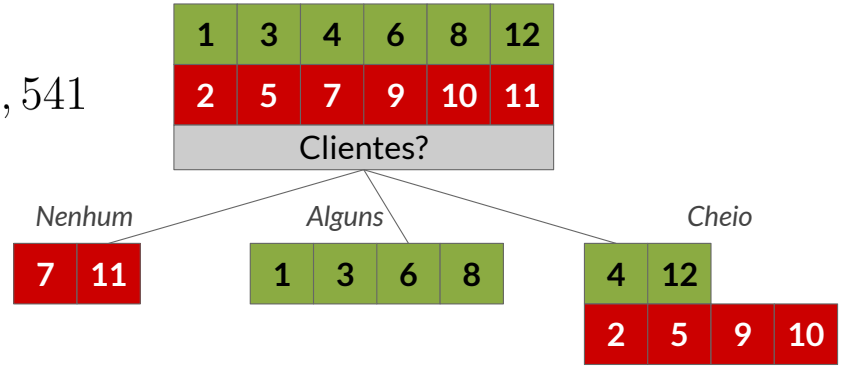


Exemplo – Esperar uma mesa?

Qual a **razão de ganho** do atributo **"Clientes"**?

$$ganho(Clientes) \approx 0,541$$

$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$

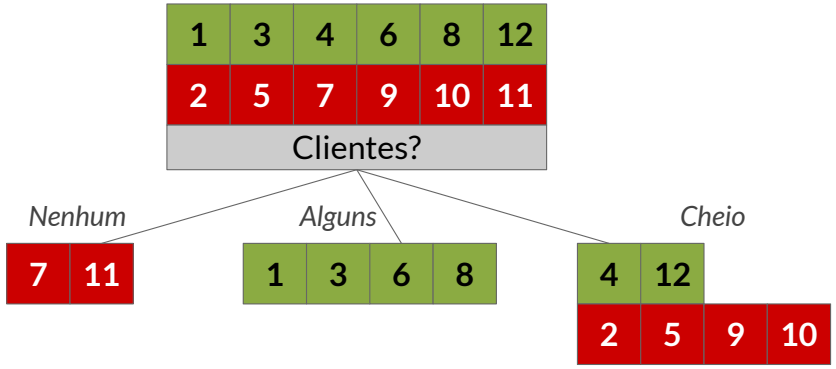


$$SplitInfo(Clientes) = -\sum_i \frac{|S_i|}{|S|} log_2 \left(\frac{|S_i|}{|S|}\right)$$

$$= -\left[\frac{2}{12} log_2 \left(\frac{2}{12}\right) + \frac{4}{12} log_2 \left(\frac{4}{12}\right) + \frac{6}{12} log_2 \left(\frac{6}{12}\right)\right] = 1,459$$

Exemplo – Esperar uma mesa?

Qual a **razão de ganho** do atributo "**Clientes**"?

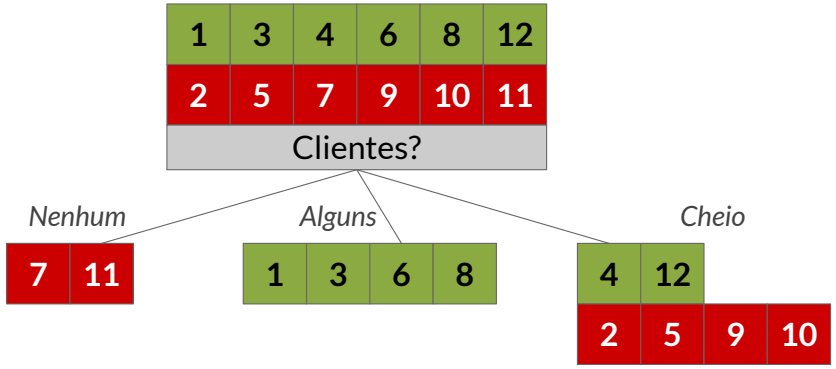


$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$

$$GainRatio(Clientes) = \frac{Ganho(Clientes)}{SplitInfo(Clientes)} = \frac{0,541}{1,459} = 0,37$$

Exemplo – Esperar uma mesa?

Qual a **razão de ganho** do atributo "**Clientes**"?



$$GainRatio(Clientes) = \frac{ganho(Clientes)}{SplitInfo(Clientes)}$$

$$GainRatio(Clientes) = \frac{Ganho(Clientes)}{SplitInfo(Clientes)} = \frac{0,541}{1,459} = 0,37$$

É a proporção
de informação gerada pela
partição que aparenta ser **útil**
para a classificação

Para não confundir

Medida	O que mede
Entropia	Incerteza das classes
Ganho de Informação	Redução na incerteza
SplitInfo	Fragmentação da divisão
Gain Ratio	Eficiência da divisão

3.
Algoritmo CART

Como funciona?

Algoritmo CART

CART – Classification and Regression Trees

Proposto por Leo Breiman, Jerome Friedman, RIchard Olshen e Charles Stone no livro “Classification and Regression Trees” (1984).

Algoritmo CART

CART – Classification and Regression Trees

- É amplamente utilizado tanto para **tarefas de classificação quanto para regressão**
- Constrói **árvores binárias** (cada nó interno tem exatamente dois filhos)
- Utiliza critérios como o **índice de Gini** (classificação) ou o **erro quadrático médio** (regressão) para determinar as divisões nos nós

Índice Gini

Mede a **impureza** de um nó

- Nó puro $\longrightarrow$ Gini = 0
- Nó misturado $\longrightarrow$ Gini alto

Ou seja, quanto maior o Gini, **mais misturadas** estão as classes

Índice Gini

Mede a **impureza** de um nó

- Nó puro $\longrightarrow$ Gini = 0
- Nó misturado $\longrightarrow$ Gini alto

Ou seja, quanto maior o Gini, **mais misturadas** estão as classes

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Índice Gini

Mede a **impureza** de um nó

- Nó puro → Gini = 0
- Nó misturado → Gini alto

Ou seja, quanto maior o Gini, **mais misturadas** estão as classes

Proporção da classe *i* no conjunto *S*

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Conjunto *S*

Índice Gini

Mede a **impureza** de um nó

- Nó puro $\longrightarrow$ Gini = 0
- Nó misturado $\longrightarrow$ Gini alto

Ou seja, quanto maior o Gini, **mais misturadas** estão as classes

$$Gini_{pond} = \sum_{i=1}^{v} \frac{|S_i|}{|S|} Gini(S_i)$$

Índice Gini

Mede a **impureza** de um nó

- Nó puro → Gini = 0
- Nó misturado → Gini alto

Ou seja, quanto maior o Gini, **mais misturadas** estão as classes

Se um atributo divide o conjunto em v subconjuntos, o Gini da divisão é uma **média ponderada**

$$Gini_{pond} = \sum_{i=1}^{v} \frac{|S_i|}{|S|} Gini(S_i)$$

Índice Gini

Mede a **impureza** de um nó

- Nó puro → Gini = 0
- Nó misturado → Gini alto

Ou seja, quanto maior o Gini, **mais misturadas** estão as classes

$$Gini_{pond} = \sum_{i=1}^{v} \frac{|S_i|}{|S|} Gini(S_i)$$

Assim como no ganho de informação, podemos calcular o ganho usando Gini

$$Ganho = Gini(S) - Gini_{pond}$$

Referências

RUSSEL, Stuart J.; NORVIG, James. **Artificial Intelligence: a modern approach.** (***Cap. 18 – 3ª ed. ou Cap. 19 – 4ª ed.***)

HSSINA, Badr et al. **A comparative study of decision tree ID3 and C4. 5.** IJACSA, v. 4, n. 2, p. 13-19, 2014.

SINGH, Sonia; GUPTA, Priyanka. **Comparative study ID3, cart and C4.5 decision tree algorithm: a survey.** IJAIST, v. 27, n. 27, p. 97-103, 2014.

Material de aula da Profa. Cristiane Nobre.